

**LAPORAN WORKSHOP SISTEM INFORMASI
WEB SERVER
(TIF330805)**

SEMESTER III



BKPM: ACARA 15

**NAMA:
SHERYN FEBRYLIA HENDRITA
(E41250604)**

**PROGRAM STUDI DILUAR KAMPUS UTAMA (PSDKU)
TEKNIK INFORMATIKA KAB. NGANJUK
JURUSAN TEKNOLOGI INFORMASI
POLITEKNIK NEGERI JEMBER
TAHUN 2026**

1. PENDAHULUAN

Praktikum Acara 15 bertujuan untuk memahami konsep Application Programming Interface (API) dan format pertukaran data JSON pada aplikasi web. Praktikum ini juga bertujuan untuk membuat API sederhana yang dapat digunakan untuk mengelola data mahasiswa dengan metode HTTP GET, POST, PUT, dan DELETE.

Selain itu, praktikum dilakukan untuk memahami cara menerima data JSON dari client, mengolah data menggunakan PHP dan database MySQL, memberikan response dalam bentuk JSON, serta melakukan pengujian API menggunakan browser dan Postman.

2. DASAR TEORI

1. Persiapan Project Acara 15

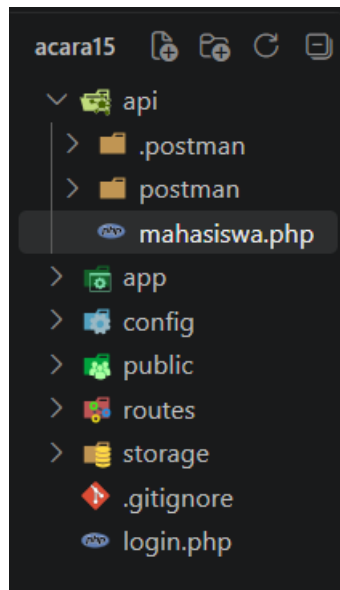
Praktikum diawali dengan membuat project baru bernama acara15 pada folder project bkpm-webserver. Project ini digunakan untuk membuat API sederhana yang berhubungan dengan data mahasiswa.

Struktur project yang digunakan adalah sebagai berikut:

```
acara15/  
├── api/  
│   └── mahasiswa.php  
├── app/  
├── config/  
├── public/  
└── storage/
```

File utama untuk API mahasiswa berada pada:

acara15/api/mahasiswa.php



2. Konfigurasi API dan JSON

Pada file mahasiswa.php, response API diatur agar menggunakan format JSON. File Database.php juga dipanggil untuk melakukan koneksi ke database.

Kode yang digunakan:

```
<?php
```

```
header('Content-Type: application/json');
```

```
require_once __DIR__ . '/../app/Core/Database.php';
```

```
try {
```

```
    $database = new Database();
```

```
    $pdo = $database->getConnection();
```

Penggunaan:

```
header('Content-Type: application/json');
```

Berfungsi untuk memberitahukan bahwa response yang diberikan oleh server menggunakan format JSON.

```

api > mahasiswa.php
1  <?php
2
3  header('Content-Type: application/json');
4
5  require_once __DIR__ . '/../app/Core/Database.php';
6
7  try {
8
9      $database = new Database();
10     $pdo = $database->getConnection();
11
12     // =====
13     // POST - Tambah Mahasiswa
14     // =====
15     if ($_SERVER['REQUEST_METHOD'] === 'POST') {
16
17         $input = json_decode(file_get_contents('php://input'), true);
18
19         if (!$input) {
20             http_response_code(400);
21
22             echo json_encode([
23                 'success' => false,
24                 'message' => 'Data JSON tidak valid'
25             ]);
26
27             exit;
28         }
29
30         $nim = $input['nim'] ?? '';
31         $nama = $input['nama'] ?? '';
32         $email = $input['email'] ?? '';
33         $prodi_id = $input['prodi_id'] ?? '';
34         $angkatan = $input['angkatan'] ?? '';
35
36         if (

```

3. Implementasi GET untuk Mengambil Seluruh Data

Method GET digunakan untuk mengambil seluruh data mahasiswa dari database.

Query yang digunakan:

```
$stmt = $pdo->query("

```

```
    SELECT id, nim, nama, email, prodi_id, angkatan

```

```
    FROM mahasiswa

```

```
    ORDER BY nim

```

```
");

```

```
$data = $stmt->fetchAll();

```

```
http_response_code(200);

```

```
echo json_encode([

```

```
    'success' => true,

```

```
'message' => 'Data mahasiswa berhasil diambil',
```

```
'data' => $data
```

```
]);
```

Data mahasiswa diambil menggunakan `fetchAll()`, kemudian dikirimkan kepada client menggunakan `json_encode()`.

API dapat diakses melalui:

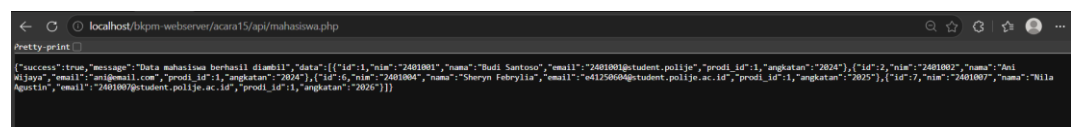
`http://localhost/bkpm-`

`webserver/acara15/api/mahasiswa.php`

```
api > mahasiswa.php
7 try {
279 // =====
280 // GET All
281 // =====
282 $stmt = $pdo->query("
283     SELECT
284     id,
285     nim,
286     nama,
287     email,
288     prodi_id,
289     angkatan
290     FROM mahasiswa
291     ORDER BY nim
292 ");
293
294 $data = $stmt->fetchAll();
295
296 http_response_code(200);
297
298 echo json_encode([
299     'success' => true,
300     'message' => 'Data mahasiswa berhasil diambil',
301     'data' => $data
302 ]);
303
304 } catch (Exception $e) {
305
306     http_response_code(500);
307
308     echo json_encode([
309         'success' => false,
310         'message' => 'Terjadi kesalahan pada server'
311     ]);
312 }
```

Hasil pengujian menunjukkan bahwa API berhasil mengambil data mahasiswa dan memberikan response dengan status 200 OK.

Gambar 4. Hasil pengujian GET seluruh data mahasiswa



```
{
  "success": true,
  "message": "Data mahasiswa berhasil diambil",
  "data": [
    {
      "id": 1,
      "nim": "2401001",
      "nama": "Budi Santoso",
      "email": "2401001@student.polije",
      "prodi_id": 1,
      "angkatan": "2024"
    },
    {
      "id": 2,
      "nim": "2401002",
      "nama": "Ani Wijaya",
      "email": "ani@email.com",
      "prodi_id": 1,
      "angkatan": "2024"
    },
    {
      "id": 3,
      "nim": "2401004",
      "nama": "Sheryn Febrylia",
      "email": "e4125060@student.polije.ac.id",
      "prodi_id": 1,
      "angkatan": "2025"
    },
    {
      "id": 7,
      "nim": "2401007",
      "nama": "Nila Agustia",
      "email": "2401007@student.polije.ac.id",
      "prodi_id": 1,
      "angkatan": "2025"
    }
  ]
}
```

4. Implementasi GET Berdasarkan ID

Selain mengambil seluruh data, API juga dapat digunakan untuk mengambil satu data mahasiswa berdasarkan ID.

Kode yang digunakan:

```
if (isset($_GET['id'])) {  
    $id = (int) $_GET['id'];  
  
    $stmt = $pdo->prepare("  
        SELECT id, nim, nama, email, prodi_id,  
angkatan  
        FROM mahasiswa  
        WHERE id = ?  
    ");  
  
    $stmt->execute([$id]);  
  
    $data = $stmt->fetch();  
    Data kemudian diperiksa. Jika data ditemukan, API  
    memberikan response berhasil.  
    http_response_code(200);  
  
    echo json_encode([  
        'success' => true,  
        'message' => 'Data mahasiswa berhasil diambil',  
        'data' => $data  
    ]);  
}
```

Pengujian dilakukan menggunakan URL:

```
http://localhost/bkpm-  
webserver/acara15/api/mahasiswa.php?id=1
```

Gambar 5. Implementasi GET berdasarkan ID

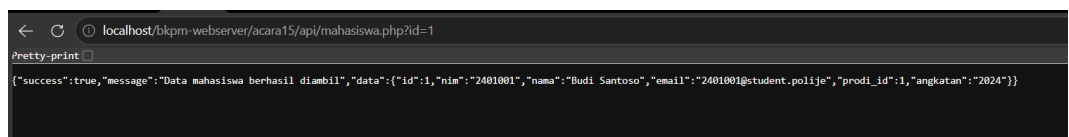
```

api > mahasiswa.php
7   try {
232 // =====
233 // GET by ID
234 // =====
235 if (isset($_GET['id'])) {
236
237     $id = (int) $_GET['id'];
238
239     $stmt = $pdo->prepare("
240         SELECT
241             id,
242             nim,
243             nama,
244             email,
245             prodi_id,
246             angkatan
247         FROM mahasiswa
248         WHERE id = ?
249     ");
250
251     $stmt->execute([$id]);
252
253     $data = $stmt->fetch();
254
255     if (!$data) {
256         http_response_code(404);
257
258         echo json_encode([
259             'success' => false,
260             'message' => 'Data mahasiswa tidak ditemukan'
261         ]);
262
263         exit;
264     }
265
266     http_response_code(200);

```

Hasil pengujian menunjukkan bahwa data mahasiswa dengan ID yang diminta berhasil ditampilkan.

Gambar 6. Hasil pengujian GET berdasarkan ID



```

localhost/bkpm-webserver/acara15/api/mahasiswa.php?id=1
pretty-print
{"success":true,"message":"Data mahasiswa berhasil diambil","data":{"id":1,"nim":"2401001","nama":"Budi Santoso","email":"2401001@student.polije","prodi_id":1,"angkatan":"2024"}}

```

5. Implementasi POST untuk Menambahkan Data

Method POST digunakan untuk menambahkan data mahasiswa baru.

Data dikirimkan dalam format JSON. Program membaca data JSON menggunakan:

```

$input = json_decode(
    file_get_contents('php://input'),

```

```
        true
```

```
    );
```

Data yang diterima kemudian disimpan ke dalam variabel:

```
$nim = $input['nim'] ?? '';
```

```
$nama = $input['nama'] ?? '';
```

```
$email = $input['email'] ?? '';
```

```
$prodi_id = $input['prodi_id'] ?? '';
```

```
$angkatan = $input['angkatan'] ?? '';
```

Sebelum disimpan, dilakukan pengecekan agar seluruh data wajib diisi.

```
if (
```

```
    empty($nim) ||
```

```
    empty($nama) ||
```

```
    empty($email) ||
```

```
    empty($prodi_id) ||
```

```
    empty($angkatan)
```

```
) {
```

```
    http_response_code(422);
```

```
    echo json_encode([
```

```
        'success' => false,
```

```
        'message' => 'Semua data wajib diisi'
```

```
    ]);
```

```
    exit;
```

```
}
```

Data kemudian dimasukkan ke database menggunakan prepared statement.

Gambar 7. Implementasi method POST untuk menambahkan data mahasiswa

```
api > mahasiswa.php
7 try {
12 // =====
13 // POST - Tambah Mahasiswa
14 // =====
15 if ($_SERVER['REQUEST_METHOD'] === 'POST') {
16
17     $input = json_decode(file_get_contents('php://input'), true);
18
19     if (!$input) {
20         http_response_code(400);
21
22         echo json_encode([
23             'success' => false,
24             'message' => 'Data JSON tidak valid'
25         ]);
26
27         exit;
28     }
29
30     $nim = $input['nim'] ?? '';
31     $nama = $input['nama'] ?? '';
32     $email = $input['email'] ?? '';
33     $prodi_id = $input['prodi_id'] ?? '';
34     $angkatan = $input['angkatan'] ?? '';
35
36     if (
37         empty($nim) ||
38         empty($nama) ||
39         empty($email) ||
40         empty($prodi_id) ||
41         empty($angkatan)
42     ) {
43         http_response_code(422);
44
45         echo json_encode([
46             'success' => false,
```

```
api > mahasiswa.php
7 try {
15 if ($_SERVER['REQUEST_METHOD'] === 'POST') {
16     empty($angkatan)
17 } {
18     http_response_code(422);
19
20     echo json_encode([
21         'success' => false,
22         'message' => 'Semua data wajib diisi'
23     ]);
24
25     exit;
26 }
27
28 $stmt = $pdo->prepare("
29     INSERT INTO mahasiswa
30     (nim, nama, email, prodi_id, angkatan)
31     VALUES (?, ?, ?, ?, ?)
32 ");
33
34 $stmt->execute([
35     $nim,
36     $nama,
37     $email,
38     $prodi_id,
39     $angkatan
40 ]);
41
42 http_response_code(201);
43
44 echo json_encode([
45     'success' => true,
46     'message' => 'Data mahasiswa berhasil ditambahkan',
47     'data' => [
48         'id' => $pdo->lastInsertId(),
49         'nim' => $nim,
50         'nama' => $nama,
```

Data JSON yang digunakan untuk pengujian:

```
{
  "nim": "2401099",
  "nama": "Mahasiswa API",
  "email": "2401099@student.polije.ac.id",
```

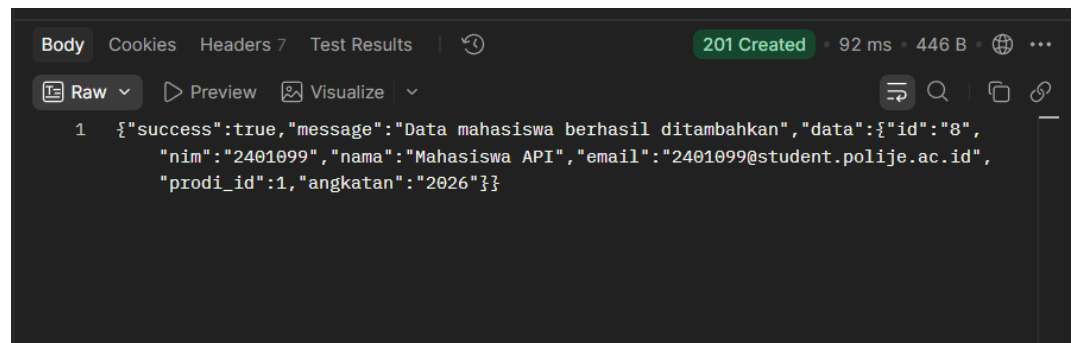
```

    "prodi_id": 1,
    "angkatan": "2026"
}

```

Hasil pengujian menunjukkan bahwa data berhasil ditambahkan dengan status 201 Created.

Gambar 8. Hasil pengujian POST penambahan data mahasiswa



6. Implementasi PUT untuk Mengubah Data

Method PUT digunakan untuk mengubah data mahasiswa yang sudah tersimpan di database.

ID mahasiswa dikirim melalui parameter URL:

`http://localhost/bkpm-`

`webserver/acara15/api/mahasiswa.php?id=1`

Data JSON yang digunakan:

```

{
    "nim": "2401001",
    "nama": "Budi Santoso Updated",
    "email": "budi.updated@student.polije.ac.id",
    "prodi_id": 1,
    "angkatan": "2024"
}

```

Data kemudian diproses menggunakan query UPDATE:

```
$stmt = $pdo->prepare("

```

```

UPDATE mahasiswa
SET nim = ?,
    nama = ?,
    email = ?,
    prodi_id = ?,
    angkatan = ?
WHERE id = ?
");

$stmt->execute([
    $nim,
    $nama,
    $email,
    $prodi_id,
    $angkatan,
    $id
]);

```

Gambar 9. Implementasi method PUT untuk mengubah data mahasiswa

```

api > mahasiswa.php
7 try {
86 // =====
87 // PUT - Update Mahasiswa
88 // =====
89 if ($_SERVER['REQUEST_METHOD'] === 'PUT') {
90     $id = $_GET['id'] ?? null;
91
92     if (!$id) {
93         http_response_code(400);
94
95         echo json_encode([
96             'success' => false,
97             'message' => 'ID mahasiswa wajib diisi'
98         ]);
99         exit;
100     }
101
102     $input = json_decode(
103         file_get_contents('php://input'),
104         true
105     );
106
107     if (!$input) {
108         http_response_code(400);
109
110         echo json_encode([
111             'success' => false,
112             'message' => 'Data JSON tidak valid'
113         ]);
114         exit;
115     }
116
117     $stmt = $pdo->prepare("
118         UPDATE mahasiswa
119         SET nim = ?,
120             nama = ?,
121             email = ?,
122             prodi_id = ?,
123             angkatan = ?
124         WHERE id = ?
125     ");
126
127     $stmt->execute([
128         $nim,
129         $nama,
130         $email,
131         $prodi_id,
132         $angkatan,
133         $id
134     ]);
135
136     echo json_encode([
137         'success' => true,
138         'message' => 'Data mahasiswa berhasil diupdate'
139     ]);
140     exit;
141 }
142
143 http_response_code(404);
144 echo json_encode([
145     'success' => false,
146     'message' => 'Metode PUT tidak valid'
147 ]);
148 exit;
149 }

```

```

api > mahasiswa.php
7 try {
90 if ($_SERVER['REQUEST_METHOD'] === 'PUT') {
121     $nim = $_input['nim'] ?? '';
122     $nama = $_input['nama'] ?? '';
123     $email = $_input['email'] ?? '';
124     $prodi_id = $_input['prodi_id'] ?? '';
125     $angkatan = $_input['angkatan'] ?? '';
126
127     if (
128         empty($nim) ||
129         empty($nama) ||
130         empty($email) ||
131         empty($prodi_id) ||
132         empty($angkatan)
133     ) {
134         http_response_code(422);
135
136         echo json_encode([
137             'success' => false,
138             'message' => 'Semua data wajib diisi'
139         ]);
140         exit;
141     }
142
143     $stmt = $pdo->prepare("
144         UPDATE mahasiswa
145         SET nim = ?,
146             nama = ?,
147             email = ?,
148             prodi_id = ?,
149             angkatan = ?
150         WHERE id = ?
151     ");
152
153     $stmt->execute([
154         $nim,
155         $nama,
156         $email,
157         $prodi_id,
158         $angkatan,
159         $id
160     ]);
161
162     echo json_encode([
163         'success' => true,
164         'message' => 'Data mahasiswa berhasil diupdate'
165     ]);
166     exit;
167 }
168
169 http_response_code(404);
170 echo json_encode([
171     'success' => false,
172     'message' => 'Metode PUT tidak valid'
173 ]);
174 exit;
175 }

```

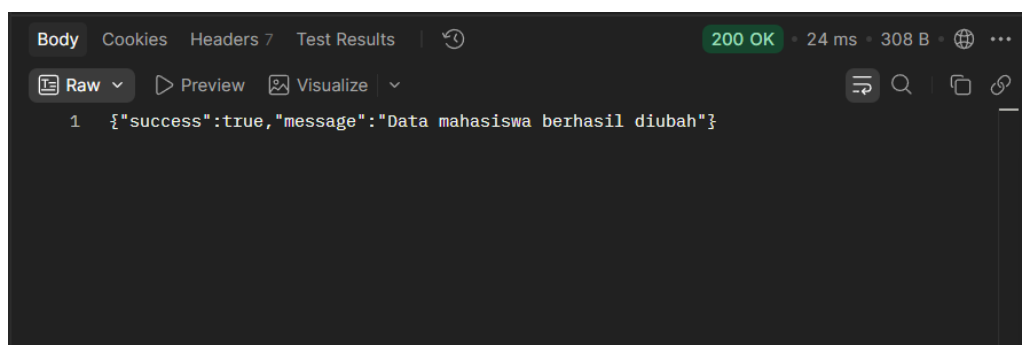
```

153
154     $stmt->execute([
155         $nim,
156         $nama,
157         $email,
158         $prodi_id,
159         $angkatan,
160         $id
161     ]);
162
163     http_response_code(200);
164
165     echo json_encode([
166         'success' => true,
167         'message' => 'Data mahasiswa berhasil diubah'
168     ]);
169
170     exit;
171 }

```

Hasil pengujian menunjukkan bahwa data mahasiswa berhasil diubah dan API memberikan status 200 OK.

Gambar 10. Hasil pengujian PUT perubahan data mahasiswa



7. Implementasi DELETE untuk Menghapus Data

Method DELETE digunakan untuk menghapus data mahasiswa berdasarkan ID.

ID mahasiswa diperoleh melalui parameter URL:

```
$id = $_GET['id'] ?? null;
```

Proses penghapusan menggunakan prepared statement:

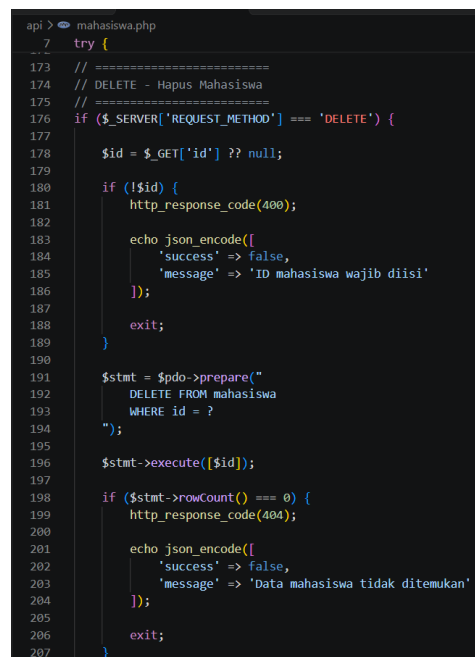
```
$stmt = $pdo->prepare("
    DELETE FROM mahasiswa
    WHERE id = ?
");
```

```
$stmt->execute([$id]);
```

Program juga melakukan pemeriksaan apakah data yang ingin dihapus tersedia.

Jika data tidak ditemukan, API memberikan response 404 Not Found.

Gambar 11. Implementasi method DELETE untuk menghapus data mahasiswa



```
api > mahasiswa.php
7 try {
173 // =====
174 // DELETE - Hapus Mahasiswa
175 // =====
176 if ($_SERVER['REQUEST_METHOD'] === 'DELETE') {
177
178     $id = $_GET['id'] ?? null;
179
180     if (!$id) {
181         http_response_code(400);
182
183         echo json_encode([
184             'success' => false,
185             'message' => 'ID mahasiswa wajib diisi'
186         ]);
187         exit;
188     }
189
190     $stmt = $pdo->prepare("
191         DELETE FROM mahasiswa
192         WHERE id = ?
193     ");
194
195     $stmt->execute([$id]);
196
197     if ($stmt->rowCount() === 0) {
198         http_response_code(404);
199
200         echo json_encode([
201             'success' => false,
202             'message' => 'Data mahasiswa tidak ditemukan'
203         ]);
204         exit;
205     }
206 }
207 }
```

```

209     http_response_code(200);
210
211     echo json_encode([
212         'success' => true,
213         'message' => 'Data mahasiswa berhasil dihapus'
214     ]);
215
216     exit;
217 }
218

```

Pengujian dilakukan menggunakan:

<http://localhost/bkpm-webserver/acara15/api/mahasiswa.php?id=1>

Hasil pengujian menunjukkan bahwa data berhasil dihapus dan API memberikan status 200 OK.

Gambar 12. Hasil pengujian DELETE data mahasiswa

```

209     http_response_code(200);
210
211     echo json_encode([
212         'success' => true,
213         'message' => 'Data mahasiswa berhasil dihapus'
214     ]);
215
216     exit;
217 }
218

```

8. Pengujian Data Mahasiswa yang Tidak Ditemukan

Pengujian berikutnya dilakukan dengan memasukkan ID yang tidak terdapat di database.

URL yang digunakan:

<http://localhost/bkpm-webserver/acara15/api/mahasiswa.php?id=9999999>

Program melakukan pengecekan:

```

if (!$data) {
    http_response_code(404);

    echo json_encode([
        'success' => false,
        'message' => 'Data mahasiswa tidak ditemukan'
    ]);
}

```

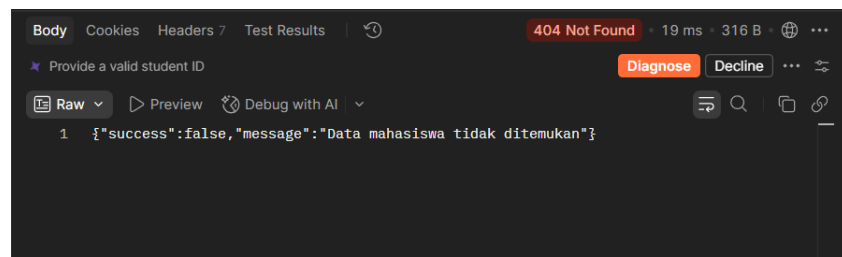
```
});
```

```
    exit;  
}
```

Hasil pengujian memberikan status 404 Not Found dengan response:

```
{  
  "success": false,  
  "message": "Data mahasiswa tidak ditemukan"  
}
```

Gambar 13. Pengujian GET dengan ID mahasiswa yang tidak ditemukan



9. Pengujian Validasi Data JSON

Pengujian validasi dilakukan dengan mengirimkan data kosong melalui method POST.

Data yang dikirim:

```
{  
  "nim": "",  
  "nama": "",  
  "email": "",  
  "prodi_id": "",  
  "angkatan": ""  
}
```

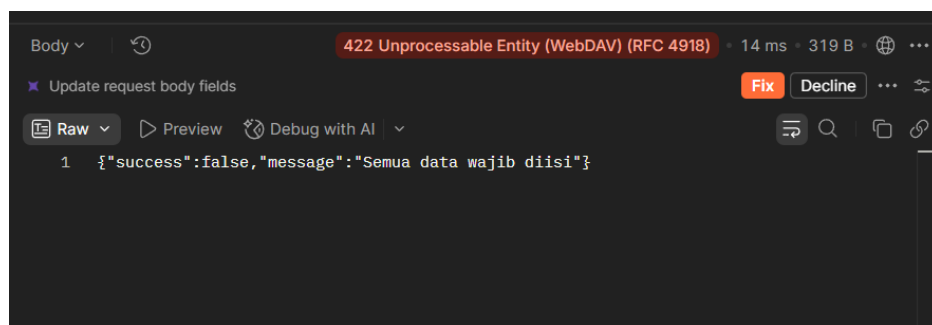
API melakukan pemeriksaan terhadap data yang diterima. Jika terdapat data yang kosong, server memberikan status 422 Unprocessable Content.

Response yang diperoleh:

```
{  
  "success": false,  
  "message": "Semua data wajib diisi"  
}
```

Hal tersebut menunjukkan bahwa API telah memiliki validasi untuk mencegah data kosong masuk ke dalam database.

Gambar 14. Pengujian validasi data JSON kosong



10. Penggunaan Prepared Statement

Pada proses pengolahan data, API menggunakan prepared statement melalui fungsi:

```
$pdo->prepare();
```

Kemudian data diberikan menggunakan:

```
$stmt->execute([
```

```
    $nim,
```

```
    $nama,
```

```
    $email,
```

```
    $prodi_id,
```

```
    $angkatan
```

```
]);
```

Prepared statement digunakan pada proses INSERT, UPDATE, DELETE, dan GET berdasarkan ID. Penggunaan prepared statement membantu membuat proses query database lebih aman karena nilai input dipisahkan dari struktur query SQL.

Contoh penerapannya:

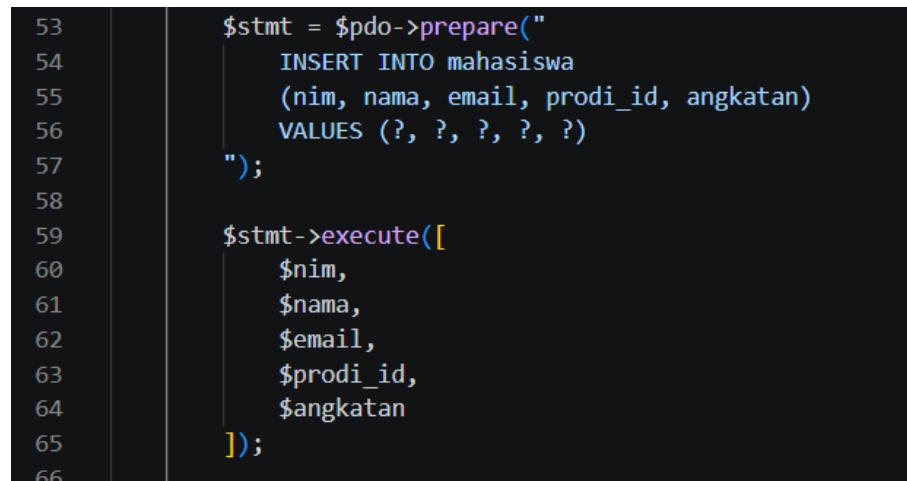
```

$stmt = $pdo->prepare("
    INSERT INTO mahasiswa
    (nim, nama, email, prodi_id, angkatan)
    VALUES (?, ?, ?, ?, ?)
");

$stmt->execute([
    $nim,
    $nama,
    $email,
    $prodi_id,
    $angkatan
]);

```

Gambar 15. Penggunaan prepared statement pada API mahasiswa



```

53      $stmt = $pdo->prepare("
54          INSERT INTO mahasiswa
55          (nim, nama, email, prodi_id, angkatan)
56          VALUES (?, ?, ?, ?, ?)
57      ");
58
59      $stmt->execute([
60          $nim,
61          $nama,
62          $email,
63          $prodi_id,
64          $angkatan
65      ]);
66

```

11. Struktur Method API

API yang dibuat pada praktikum ini memiliki beberapa method utama, yaitu:

Method	Fungsi
GET	Mengambil seluruh data mahasiswa
GET + ID	Mengambil satu data mahasiswa berdasarkan ID

POST	Menambahkan data mahasiswa
PUT	Mengubah data mahasiswa
DELETE	Menghapus data mahasiswa

Dengan demikian, API yang dibuat telah dapat menangani operasi CRUD (Create, Read, Update, Delete) melalui HTTP method.

Pengujian dilakukan menggunakan browser untuk GET dan Postman untuk menguji request dengan berbagai HTTP method serta data JSON.

Kesimpulan

Berdasarkan praktikum Acara 15, dapat disimpulkan bahwa API digunakan sebagai perantara antara client dan server untuk melakukan pertukaran data. Data yang dikirim dan diterima pada API yang dibuat menggunakan format JSON sehingga dapat diproses dengan mudah oleh aplikasi client.

API mahasiswa yang dibuat telah berhasil menerapkan operasi CRUD menggunakan method GET, POST, PUT, dan DELETE. Method GET digunakan untuk mengambil data, POST untuk menambahkan data, PUT untuk mengubah data, dan DELETE untuk menghapus data.

Selain itu, API juga telah menerapkan validasi terhadap data yang diterima serta memberikan HTTP status code yang sesuai, seperti 200 OK, 201 Created, 404 Not Found, dan 422 Unprocessable Content. Penggunaan prepared statement pada query database juga diterapkan untuk meningkatkan keamanan dalam pengolahan data.

Dengan praktikum ini, konsep dasar REST API, JSON, HTTP method, response status, validasi input, dan komunikasi antara client dengan server dapat dipahami dan diterapkan secara langsung.